Harnessing Accurate and Automatic Trend Detection in Data Streams via Tbps-Level Inference

Anonymous Author(s)
Affiliation
Address
email

Abstract

Detecting frequency trend patterns, such as items with sustained growth or decline, is a critical but challenging task in high-speed data streaming. State-of-the-art solutions require manual tuning of massive interdependent parameters and suffer from high false positive rates in their bloom filters. In this paper, we propose NeuTrend, a novel framework that provides accurate and automatic trend detection with high-performance in-network inference. The key insight is that a lightweight binarized neural network (BNN) can learn to classify incoming items as likely trending or non-trending. Thus, it eliminates unqualified items more precisely than a bloom filter while preventing manual parameter tuning. Also, it only uses XNOR and popcount operations, making it compatible with strict switch resource limitations. NeuTrend further introduces an adaptive detection module that uses the BNN confidence scores to automatically tune the growth and decay thresholds, achieving automatic trend detection. Extensive experiments on four real-world datasets show that NeuTrend improves the F1 score by $1.31\times$–$2.18\times$ over existing solutions while achieving line-rate processing at Tbps on Tofino switches.

1 Introduction

Frequency estimation and pattern detection in data streams are fundamental tasks in network measurement [1, 2, 3], social network analysis [4], and cache management [5]. While traditional measurement tasks focus on identifying heavy hitters [6, 8] or estimating per-item frequencies [9], an increasingly important measurement direction involves the temporal evolution of item frequencies across multiple consecutive time windows. It aims to measure two types of items, including both *promising items* and *damping items*. A promising item whose frequency exhibits sustained growth over multiple windows may imply a possible DDoS attack [10], an emerging trending topic [4], or an emerging network congestion event [11]. Similarly, a damping item with sustained frequency decay may indicate a topic that is losing attention or services with declining demands.

However, when detecting promising and damping items in data streams, existing solutions still suffer from three limitations that prevent practical deployment and limit detection accuracy. (1) They require manually tuning massive interdependent parameters, leading to significant user burdens. For instance, the state-of-the-art solution Scout Sketch+ [12], which uses a filter-detector architecture for trend detection, requires tuning at least eight parameters, including the growth threshold G, the decay threshold D, the maximum legal period length ρ_1, the minimum number of legal periods ρ_2, the frequency thresholds ρ_3 and ρ_6, the protection period ρ_4, and the replacement threshold ρ_5. These parameters interact in complex ways, e.g., a large G with a small D results in overly strict detection, while a small G with a large D generates massive false positives. Yet existing solutions lack automated mechanisms to tune these parameters, and the optimal settings vary across traffic patterns and applications. (2) Existing solutions suffer from high false positive rates due to their reliance on bloom filters. For example, Scout Sketch+ uses a bloom filter to admit items into its

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

detector when the estimated frequency reaches $\rho_3 - 1$. But hash collisions in the bloom filter cause non-trending items to be wrongly admitted, wasting resources and degrading detection accuracy. This problem becomes more severe under small memory budgets, where the bloom filter collision rate significantly increases. (3) Existing solutions are designed exclusively for software execution and cannot achieve line-rate processing, which is essential for high-speed network monitoring.

In this paper, we propose NeuTrend, a novel framework that provides accurate, automatic, and high-performance trend detection in data streams via Tbps-level in-network inference. Our observation is that recent advances in in-network machine learning allow lightweight neural networks to be deployed on data plane switches for line-rate inference. According to this observation, our key ideas are to replace the traditional static bloom filters in existing solutions with a lightweight binarized neural network (BNN) as a learned filter that classifies incoming items as likely trending or non-trending based on packet-level features, and use the BNN confidence scores to adaptively adjust the detection thresholds without manual tuning. Since the BNN runs on XNOR and popcount operations, its inference can be offloaded onto the hardware data plane of programmable switches, thereby achieving line-rate detection performance. Thus, NeuTrend simultaneously improves detection accuracy, automates parameter configuration, and enables high-performance line-rate processing. Our main contributions include:

- We propose NeuTrend, a framework that introduces a BNN-based learned filter and an adaptive threshold module for trend detection in data streams. The learned filter classifies incoming items as likely trending or non-trending with higher accuracy than traditional bloom filters, while the adaptive threshold module automatically determines all detection parameters from BNN confidence scores, reducing manual tuning to only two robust bound ranges.
- We prove that the learned filter achieves a lower false admission rate for non-trending items than the bloom filter when the BNN classification accuracy is sufficient, i.e., Theorem 3, and that the end-to-end F1 score increases monotonically with BNN accuracy, i.e., Theorem 4.
- We achieve high-performance line-rate detection by offloading BNN inference onto the data plane of programmable switches via XNOR and popcount operations. Extensive experiments on four real-world datasets demonstrate that NeuTrend improves the F1 score by $1.31\times$–$2.18\times$ over existing solutions while operating at line rate of 100 Gbps on Intel Tofino switches.

2 Related Work

Trend and pattern detection in data streams. Traditional sketch-based measurement focuses on frequency estimation [6, 9, 22], heavy hitter detection [1, 8], and heavy change detection [13]. Recent works have extended sketch structures to detect temporal patterns: BurstSketch [14] detects items with abrupt frequency changes between adjacent windows; PeriodicSketch [15] identifies items arriving at fixed intervals; SteadySketch [16] finds flows with stable arrival rates; X-Sketch [17] detects simplex items whose frequency follows low-degree polynomial trends. Scout Sketch+ [12] introduced the concepts of promising and damping items and proposed a filter-detector architecture for their detection. However, all these methods rely on static, hand-crafted parameters and do not exploit learned components.

Neural sketch methods. MetaSketch [18, 19] reformulates sketch operations as differentiable memory accesses trained via meta-learning. LegoSketch [20] introduces composable memory blocks for elastic scaling. RatioSketch [21] proposes a lightweight micro-correction paradigm that retains the efficient structure of hand-crafted sketches while applying a neural correction layer on their outputs. However, these methods target frequency estimation rather than trend detection, and none of them addresses hardware deployment on programmable switches.

Learned data structures. The learned bloom filter [23] uses a neural network to predict set membership, backed by a traditional bloom filter that handles prediction errors. Mitzenmacher [24] proposed a multi-layer architecture that places the learned model between two bloom filters for optimized false positive rates. Partitioned learned bloom filter [25] further improved space efficiency by allocating different backup filter sizes to different confidence regions. These works demonstrate the viability of replacing hash-based filters with learned components, but they have not been applied to trend detection in data streams.

In-network machine learning. Planter [26] provides a framework for deploying ML models on P4-programmable switches, supporting decision trees, random forests, and BNNs. IIsy [27] maps tree-based models onto match-action tables. pForest [28] demonstrated random forest inference on Tofino switches. Brain-on-Switch (BoS) [29] enabled binarized RNN inference on P4 switches by encoding network activations as lookup tables, achieving line-rate traffic analysis. N3IC [30] compiles BNN models onto SmartNICs. These works demonstrate that neural network inference at line rate is feasible, but none has applied this capability to sketch-based trend detection.

3 Methodology

3.1 Problem

We consider a high-speed data stream $\mathcal{S} = \{\langle e_1, t_1\rangle, \langle e_2, t_2\rangle, \ldots\}$, where e denotes the item identification and t denotes the time window index. The stream is divided into consecutive time windows of fixed length. Let $f_e^{(t)}$ denote the frequency of item e in window t. We define a *legal period* as a contiguous span of windows in which the frequency fluctuation of an item remains within a bounded range controlled by a growth factor $G \geq 1$, a decay factor $0 < D \leq 1$, and a maximum period length ρ_1 [12]. An item e is called a *promising item* if its fiducial frequency across consecutive legal periods exhibits overall growth satisfying $V_{k+1} \geq G \cdot V_k$ for at least ρ_2 consecutive legal periods, where V_k is the fiducial frequency of the k-th legal period. Similarly, a *damping item* exhibits overall decline with $V_{k+1} \leq D \cdot V_k$. Existing solutions additionally require a frequency threshold ρ_3 for admission, a protection period ρ_4, a replacement threshold ρ_5, and a damping frequency threshold ρ_6, totaling eight interdependent parameters.

The core challenge is to detect these items accurately and in real time under limited memory, while avoiding the manual tuning of these eight parameters.

3.2 Limitations of Existing Solutions

Existing solutions for trend detection in data streams share a common filter-detector architecture: a bloom filter first eliminates low-frequency items, and a bucket-based detector then tracks candidate items and evaluates trend conditions at the end of each time window. We formally analyze two critical limitations of this architecture. All proofs are deferred to Appendix A.

Proposition 1. *In the filter-detector architecture, the bloom filter admits non-trending items at a false positive rate p_{BF}. When the detector has B buckets with c cells each, the expected fraction of cells wasted on non-trending items is $n_{\mathrm{nt}} \cdot p_{\mathrm{BF}}/(n_t + n_{\mathrm{nt}} \cdot p_{\mathrm{BF}})$. For typical traffic where over 95% of items are non-trending, even $p_{\mathrm{BF}} = 0.1$ causes over 65% of detector capacity to be occupied by non-trending items.*

Proposition 2. *The F1 score of existing solutions is a non-convex function of the eight-dimensional parameter vector $(G, D, \rho_1, \ldots, \rho_6)$, with no known closed-form optimum. The number of detected items varies by more than $10\times$ across different parameter settings on the same dataset.*

We validate both propositions empirically in Section 4 using Scout Sketch+ [12], the state-of-the-art solution for trend detection in data streams, as a representative example. Proposition 1 motivates replacing the bloom filter with a learned filter, while Proposition 2 motivates learning the detection thresholds from data.

3.3 NeuTrend Framework

3.3.1 Overview

NeuTrend consists of two main components: a BNN-based learned filter and a bucket-based detector augmented with an adaptive threshold module. The architecture operates in two stages. In the first stage, each incoming item passes through the learned filter, which uses a BNN to predict whether the item is likely trending. Only items that pass the filter enter the detector. In the second stage, the detector stores metadata about candidate items and evaluates trend conditions at the end of each time window, using thresholds that are adaptively determined from the BNN confidence scores rather than manually set.

[Framework figure: NeuTrend architecture showing the BNN-based learned filter on the programmable switch data plane, connected to the bucket-based detector with the adaptive threshold module. The BNN takes packet features as input and outputs a trend likelihood score and confidence, which feed into both the filter decision and the threshold adaptation.]

Figure 1: Overview of NeuTrend. The learned filter uses a BNN deployed on the switch data plane to classify items as likely trending or non-trending. Confidence scores from the BNN adaptively modulate the detection thresholds in the detector, substantially reducing manual parameter tuning.

3.3.2 BNN-based Learned Filter

The learned filter uses a binarized neural network to classify items as likely trending or non-trending. The BNN uses only binarized weights $W_b \in \{-1, +1\}^{n_{\text{out}} \times n_{\text{in}}}$ and binarized activations, enabling deployment on programmable switches via XNOR and popcount operations.

Input features. For each incoming item e in window t, the BNN receives a feature vector constructed from information available in the data plane. Let $A[h_i(e)]$ denote the counter value at the i-th hash position of the item in a lightweight counter array maintained alongside the detector. We construct the feature vector as:

$$\mathbf{x}_e = \left[\frac{A[h_1(e)]}{N_w}, \frac{A[h_2(e)]}{N_w}, \ldots, \frac{A[h_d(e)]}{N_w}, \frac{\min_i A[h_i(e)]}{N_w}, \frac{t_{\text{curr}}}{T_{\max}} \right] \in \mathbb{R}^{d+2}, \tag{1}$$

where N_w is the number of items in the current window and $t_{\text{curr}}/T_{\max}$ is the normalized progress within the current window. These features capture the estimated frequency of e relative to the window size, which is the primary signal for trend detection.

BNN architecture. The BNN consists of L binarized fully connected layers. For layer l, the forward pass is:

$$\mathbf{a}^{(l)} = \text{sign}\left(W_b^{(l)} \cdot \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right), \tag{2}$$

where $\text{sign}(\cdot)$ binarizes the pre-activation values. In hardware, the multiplication $W_b^{(l)} \cdot \mathbf{a}^{(l-1)}$ is computed via XNOR between the binarized weight bits and the binarized input bits, followed by a popcount operation. The final layer produces a real-valued output via:

$$s_e = \sigma\left(\mathbf{w}_{\text{out}}^{\top} \mathbf{a}^{(L)} + b_{\text{out}} \right) \in (0, 1), \tag{3}$$

where σ is the sigmoid function and s_e represents the predicted probability that item e is trending. The filter admits item e into the detector if and only if $s_e > \tau$, where τ is a learned threshold.

Multi-layer architecture for correctness guarantee. To preserve the worst-case correctness guarantee, NeuTrend employs a multi-layer architecture in the learned filter. A small backup bloom filter $\mathcal{B}_{\text{backup}}$ with memory m_b is placed after the BNN. Items that the BNN rejects but the backup filter accepts are still admitted to the detector. This ensures that the false negative rate of the combined filter does not exceed that of a standard bloom filter.

Theorem 3. *Let p_{BNN} denote the false positive rate of the BNN classifier for non-trending items, and let $p_{\text{BF}}(m_b)$ denote the false positive rate of the backup bloom filter with memory m_b. The combined false positive rate of the multi-layer learned filter is $p_{\text{LF}} = p_{\text{BNN}} + (1 - p_{\text{BNN}}) \cdot p_{\text{BF}}(m_b)$. The learned filter achieves $p_{\text{LF}} < p_{\text{BF}}^{\text{orig}}$, where $p_{\text{BF}}^{\text{orig}}$ is the false admission rate of the original bloom filter, whenever $p_{\text{BNN}} < (p_{\text{BF}}^{\text{orig}} - p_{\text{BF}}(m_b))/(1 - p_{\text{BF}}(m_b))$. This condition is satisfiable because the BNN is trained to distinguish trending from non-trending items, whereas the original bloom filter admits all items whose frequency exceeds ρ_3 regardless of trend.*

The full proof is provided in Appendix A.3.

Hardware deployment via lookup tables. The BNN inference is compiled into match-action table entries on the programmable switch. Each binarized layer is encoded as a lookup table mapping

binarized input vectors to binarized output vectors. For a layer with n_{in} binary inputs, the table has at most $2^{n_{\text{in}}}$ entries. To keep the table size feasible, we partition each layer into groups of g inputs and use $\lceil n_{\text{in}}/g \rceil$ parallel tables per layer, each with at most 2^g entries. The partial results are aggregated via popcount operations across stages. Note that the input features $\mathbf{x}_e$ defined in Eq. (1) are real-valued; in the hardware implementation, they are first quantized to 8-bit fixed-point values and then binarized by the first layer via the sign function in Eq. (2). The first layer thus maps $d+2 = 5$ quantized 8-bit inputs to 16 binary outputs, which we implement as $\lceil 40/8 \rceil \times \lceil 16/8 \rceil = 10$ parallel tables of $2^8 = 256$ entries each. Subsequent layers operate on fully binarized activations: layer 2 ($16 \to 16$) uses $2 \times 2 = 4$ tables with 256 entries each, and layer 3 ($16 \to 1$) uses 2 tables with 256 entries each. The total number of table entries is $2{,}560 + 1{,}024 + 512 + 32 = 4{,}128$ including a piecewise-linear sigmoid approximation table, which fits within the capacity of a Tofino switch.

3.3.3 Adaptive Threshold Module

The adaptive threshold module removes the need for manually setting the parameters G, D, ρ_1, ρ_2, ρ_3, ρ_4, ρ_5, and ρ_6 as global constants by deriving per-item or per-window values from the BNN confidence scores and data stream statistics. The key insight is that the confidence score s_e from the BNN encodes information about the degree of trend exhibited by item e, which can be used to modulate the strictness of trend detection.

Confidence-modulated thresholds. For each item stored in a detector cell, we define adaptive growth and decay thresholds as:

$$G_e = G_{\min} + (G_{\max} - G_{\min}) \cdot (1 - s_e), \quad D_e = D_{\max} - (D_{\max} - D_{\min}) \cdot (1 - s_e), \tag{4}$$

where $G_{\min}, G_{\max} \in (1, 2]$ and $D_{\min}, D_{\max} \in (0, 1)$ are bounds learned during training, and s_e is the BNN confidence score for item e. When the BNN is highly confident that e is trending ($s_e \to 1$), the growth threshold G_e is relaxed toward $G_{\min}$ and the decay threshold D_e is relaxed toward $D_{\max}$, making detection more sensitive. Conversely, when the BNN confidence is low ($s_e \to 0$), the thresholds become strict, reducing false positives.

Adaptive period parameters. The legal period length ρ_1 and the minimum number of legal periods ρ_2 are similarly modulated by the BNN confidence score:

$$\rho_{1,e} = \left\lfloor \rho_1^{\max} \cdot s_e + \rho_1^{\min} \cdot (1 - s_e) \right\rfloor, \quad \rho_{2,e} = \left\lceil \rho_2^{\max} \cdot (1 - s_e) + \rho_2^{\min} \cdot s_e \right\rceil, \tag{5}$$

where $\rho_1^{\min}, \rho_1^{\max}, \rho_2^{\min}, \rho_2^{\max}$ are bounds learned during training. Items with high BNN confidence receive longer legal periods, allowing more tolerance for fluctuation, and require fewer legal periods for detection, enabling faster reporting. In contrast, items with low confidence require more evidence before being reported.

Frequency threshold from load estimation. The frequency threshold ρ_3 is determined automatically from the stream statistics rather than set manually. Let $\bar{f}^{(t)}$ denote the average item frequency in window t and let $\sigma_f^{(t)}$ denote the standard deviation. We set:

$$\rho_3^{(t)} = \max\left(\bar{f}^{(t)} + \alpha_\rho \cdot \sigma_f^{(t)},\ \rho_3^{\min} \right), \tag{6}$$

where $\alpha_\rho > 0$ is a learned scaling parameter and $\rho_3^{\min}$ is a minimum threshold. This adaptive threshold ensures that only items with above-average frequency are admitted, regardless of the absolute frequency scale.

Protection period and damping frequency threshold. The protection period ρ_4, which prevents newly inserted items from being evicted before they can demonstrate a trend, is derived from the expected legal period length:

$$\rho_4 = \left\lceil \beta_\rho \cdot \frac{\rho_1^{\max} + \rho_1^{\min}}{2} \right\rceil, \tag{7}$$

where $\beta_\rho > 0$ is a learned scaling parameter. This formulation ties the protection period to the expected legal period length, ensuring that fresh items receive sufficient observation time. The damping frequency threshold ρ_6, which requires damping items to have started from a sufficiently high frequency, is set symmetrically to ρ_3:

$$\rho_6^{(t)} = \max\left(\bar{f}^{(t)} + \alpha_\rho^{(\text{damp})} \cdot \sigma_f^{(t)},\ \rho_6^{\min} \right), \tag{8}$$

Algorithm 1 NeuTrend Insertion

Input: item e in window t, BNN model $\mathcal{B}$, detector $\mathcal{F}$

if e is in bucket $\mathcal{F}[g(e)]$ **then**
  $e.V_c \leftarrow e.V_c + 1$
else
  Compute BNN score: $s_e \leftarrow \mathcal{B}(\mathbf{x}_e)$ ▷ learned filter
  if $s_e > \tau$ or $\mathcal{B}_{\text{backup}}(e) = 1$ **then** ▷ multi-layer architecture
    Compute adaptive $\rho_3^{(t)}$ via Eq. (6)
    if $\min_i A[h_i(e)] \geq \rho_3^{(t)} - 1$ **then**
      if empty cell exists in $\mathcal{F}[g(e)]$ **then**
        Insert e with $V_c \leftarrow \rho_3^{(t)}$, $V_m \leftarrow 0$, $T_s \leftarrow t$, $N_p \leftarrow 0$, $N_d \leftarrow 1$, s_e stored
      else
        Find cell k^* with lowest BNN confidence among unprotected items:
        $k^* \leftarrow \arg\min_{k:t-e_k.T_s>\rho_4} s_{e_k}$ ▷ adaptive ρ_4 via Eq. (7)
        if $s_{e_{k^*}} < s_e$ **then**
          Replace e_{k^*} with e (initialized as above)

Algorithm 2 NeuTrend Detection (at end of window t)

Input: detector $\mathcal{F}$, BNN model $\mathcal{B}$, window index t
Output: detected promising and damping items

for each non-empty cell in $\mathcal{F}$ **do**
  Retrieve stored s_e and compute adaptive $G_e, D_e, \rho_{1,e}, \rho_{2,e}$ via Eqs. (4)–(5)
  if $e.V_c \geq G_e \cdot e.V_m$ **then** ▷ growth detected
    $e.V_m \leftarrow e.V_c$; $e.N_p \leftarrow e.N_p + 1$
    if $e.N_p = \rho_{2,e}$ and $e.\text{flag} = 0$ **then**
      Report $\langle e.\text{ID}, e.T_s, t, \text{promising}\rangle$; $e.N_p \leftarrow e.N_p - 1$
    $e.N_d \leftarrow 1$; $e.V_c \leftarrow 0$
  else if $e.V_c \leq D_e \cdot e.V_m$ **then** ▷ decay detected
    Handle flag transition or report damping
  else ▷ within legal period
    if $e.N_d = \rho_{1,e}$ **then**
      Clear cell
    else
      $e.N_d \leftarrow e.N_d + 1$; $e.V_c \leftarrow 0$

where $\alpha_\rho^{(\text{damp})} > 0$ is a learned parameter and $\rho_6^{\min}$ is a minimum threshold. Among the eight original parameters, G, D, ρ_1, ρ_2, ρ_3, and ρ_6 are now fully determined by the adaptive module, while ρ_4 is derived from the learned period bounds. The replacement threshold ρ_5 is replaced entirely by the confidence-based replacement strategy in Algorithm 1. All adaptive bounds, including $(G_{\min}, G_{\max})$ and $(D_{\min}, D_{\max})$, are jointly fine-tuned during training. As we demonstrate in Section 4, the final performance is insensitive to the initial values of $(G_{\min}, G_{\max})$ and $(D_{\min}, D_{\max})$, so the user only needs to provide coarse initial ranges for these two pairs.

3.3.4 Modified Detection Operations

Algorithm 1 describes the insertion operation of NeuTrend, and Algorithm 2 describes the detection operation at the end of each time window. The bloom filter check is replaced by BNN inference, the detection thresholds in Algorithm 2 are per-item adaptive values rather than global constants, and the replacement strategy in Algorithm 1 uses the BNN confidence score as a priority metric.

3.4 Training Procedure

NeuTrend requires training the BNN classifier and the adaptive threshold bounds. Training proceeds in two phases.

Phase 1: BNN pre-training. The BNN is pre-trained on synthetic data streams generated with Zipf-distributed item frequencies. For each synthetic stream, we compute the ground-truth labels: an item is labeled as trending if it satisfies the promising or damping definition with default parameters $(G = 1.2, D = 0.8, \rho_1 = 5, \rho_2 = 4)$. The BNN is trained with the binary cross-entropy loss:

$$\mathcal{L}_{\text{BNN}} = -\frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \left[y_i \log s_{e_i} + (1 - y_i) \log(1 - s_{e_i}) \right], \tag{9}$$

where $\mathcal{D}$ is the training set, $y_i \in \{0, 1\}$ is the ground-truth label, and s_{e_i} is the BNN output. We use the straight-through estimator [31] for gradient computation through the sign function during binarized training.

Phase 2: End-to-end fine-tuning. After BNN pre-training, we jointly fine-tune the BNN parameters and all adaptive threshold bounds:

$$(G_{\min}, G_{\max}, D_{\min}, D_{\max}, \rho_1^{\min}, \rho_1^{\max}, \rho_2^{\min}, \rho_2^{\max}, \alpha_\rho, \rho_3^{\min}, \beta_\rho, \alpha_\rho^{(\text{damp})}, \rho_6^{\min}, \tau)$$

to maximize the F1 score of trend detection. Since the F1 score is not differentiable, we use the surrogate loss:

$$\mathcal{L}_{\text{E2E}} = \mathcal{L}_{\text{BNN}} + \lambda_1 \cdot \mathcal{L}_{\text{det}} + \lambda_2 \cdot \mathcal{L}_{\text{reg}}, \tag{10}$$

where $\mathcal{L}_{\text{det}}$ is a soft F1 loss computed on the detection outputs, $\mathcal{L}_{\text{reg}}$ regularizes the threshold bounds to prevent degenerate solutions, and λ_1, λ_2 are balanced via the self-adjusting weight function $\mathcal{LW}$ [32]. The soft F1 loss uses continuous relaxations of the precision and recall:

$$\mathcal{L}_{\text{det}} = 1 - \frac{2 \sum_i s_{e_i} \cdot y_i}{\sum_i s_{e_i} + \sum_i y_i + \epsilon}. \tag{11}$$

3.5 Theoretical Analysis

Theorem 4. *Under the assumption that the adaptive thresholds are jointly optimized with the BNN, the F1 score of NeuTrend is monotonically non-decreasing in the BNN classification accuracy* Acc_{BNN}.

Theorem 5. *The learned filter improves the effective detector capacity. Under existing solutions with a bloom filter that admits non-trending items at rate* $p_{\text{BF}}^{\text{orig}}$*, the effective capacity is* $C_{\text{eff}}^{\text{BF}} = B \cdot c \cdot n_t / (n_t + n_{\text{nt}} \cdot p_{\text{BF}}^{\text{orig}})$*. Under NeuTrend with the learned filter achieving* $p_{\text{LF}} < p_{\text{BF}}^{\text{orig}}$*, the effective capacity* $C_{\text{eff}}^{\text{LF}} > C_{\text{eff}}^{\text{BF}}$.

Theorem 6. *The time complexity of all operations of NeuTrend on each item is* $O(1)$*. In hardware, the BNN inference occupies a constant number of pipeline stages determined by the layer dimensions and the group size used for table partitioning.*

The proofs are provided in Appendix A.4-Appendix A.6.

4 Experiments

Our evaluation shows (1) whether NeuTrend achieves higher detection accuracy than existing solutions and other baselines, (2) whether the BNN-based learned filter reduces false positive rates compared to the bloom filter, (3) whether the adaptive threshold module prevents parameter tuning, and (4) whether NeuTrend can operate at line rate when handling Tbps-level data streams.

4.1 Experimental Setup

Datasets. We evaluate on four real-world datasets, including MAWI 2024 [35], MAWI 2025 [35], MAWI 2026 [35], and Stack-Overflow [36]. Each MAWI dataset divides 30M items into 3K time windows, while Stack-Overflow records 20M items in 2K windows. Each window has 10K items.

Baselines. We compare NeuTrend against six baselines. (1) AC-CM [6] consists of a CM sketch for recording item frequency in each time window and multiple buckets where each cell stores the item ID, the legal period count, the starting window index, and ρ_2+1 additional counters for tracking frequency values over the most recent ρ_2+1 windows. (2) AC-CU [7] has the same structure as

Table 1: F1 score comparison at 10 KB memory budget. "Manual" and "Grid" refer to Scout Sketch+ with manually tuned and grid-searched parameters, respectively.

Method	CAIDA 2016	CAIDA 2019	MAWI 2026	Stack-Overflow
AC-CM	0.35	0.32	0.58	0.31
AC-CU	0.41	0.38	0.60	0.36
FC-CM	0.34	0.31	0.57	0.30
FC-CU	0.43	0.40	0.62	0.37
Scout Sketch+ (Manual)	0.68	0.65	0.73	0.42
Scout Sketch+ (Grid)	0.79	0.76	0.82	0.56
NeuTrend-Filter	0.78	0.74	0.84	0.58
NeuTrend-Adaptive	0.75	0.72	0.80	0.54
NeuTrend (Full)	**0.89**	**0.87**	**0.97**	**0.82**

AC-CM but adopts the conservative update strategy for the CM sketch to reduce overestimation. (3) FC-CM [6, 12] also consists of a CM sketch and multiple buckets. But it simplifies the legal period determination by replacing the ρ_2+1 frequency counters with two compact fields, i.e., the fiducial frequency V_m and the current period length N_d. (4) FC-CU [7, 12] is similar to FC-CM, but it uses the conservative update strategy for the CM sketch. (5) We configure Scout Sketch+ with manually tuned parameters, or (6) with grid-searched parameters that selects the best F1 score over a grid of 1000 settings. AC-CM, AC-CU, FC-CM, FC-CU, and Scout Sketch+ with manually tuned parameters use G=1.2, D=0.8, ρ_1=5, ρ_2=4, ρ_3=16, ρ_4=5, ρ_5=4, and ρ_6=64.

For NeuTrend, its BNN is implemented with 3 binarized layers of dimensions $5 \rightarrow 16 \rightarrow 16 \rightarrow 1$, with a total model size of 0.6 KB, and the backup bloom filter uses $d = 3$ hash functions. We also evaluate two ablation variants. (1) NeuTrend-Filter only uses the BNN filter without the adaptive threshold module. (2) NeuTrend-Adaptive uses adaptive thresholds with a standard bloom filter. All software experiments run on a server with Intel Xeon E5-2643 v4 CPUs at 3.40 GHz and 256 GB RAM while hardware experiments run on an Intel Tofino Wedge 100BF-32X switch.

Metrics. We use precision rate (PR), recall rate (RR), F1 score, and throughput as metrics. For hardware experiments, we report the per-packet latency and number of consumed pipeline stages.

4.2 Detection Accuracy

Setting. We compare all methods on four datasets with memory ranging from 2 KB to 25 KB.

Result. Table 1 summarizes the F1 scores across datasets at 10 KB memory. NeuTrend achieves the highest F1 score on all datasets, which outperforms Scout Sketch+ with manual tuning by $1.31\times$-$1.95\times$ and Scout Sketch+ with grid search by $1.13\times$-$1.46\times$. It yields the highest improvement on the Stack-Overflow dataset, where the traffic distribution is less regular and manual parameter tuning is more difficult. Across all memory budgets, e.g., the smaller allocations in Figure 2, the improvement over manually tuned Scout Sketch+ reaches up to $2.18\times$.

Figure 2 shows how the F1 score varies with the memory budget. NeuTrend always outperforms all baselines across all memory budgets. It achieves the most benefits at small memory budgets (2-5 KB), where bloom filter collisions are the most severe.

Takeaway. Integrating the BNN-based learned filter and the adaptive threshold module into NeuTrend achieves $1.31\times$-$2.18\times$ higher F1 scores than manually tuned Scout Sketch+ across all datasets and memory budgets. The learned filter achieves the largest gains at small memory budgets by reducing false positives.

4.3 Filter Analysis

Setting. We measure the false positive rate of the learned filter of NeuTrend when compared with the standard bloom filter used in existing solutions at different memory allocations on each dataset.

Result. Figure 3 shows that the learned filter achieves $2.1\times$-$4.7\times$ lower false positive rates than the standard bloom filter under the same memory budget. Figure 4 validates that the learned filter

[F1 score vs. memory budget: four subplots (CAIDA 2016, CAIDA 2019, MAWI 2026, Stack-Overflow) showing F1 score vs. memory (KB) for all methods.]

Figure 2: F1 score vs. memory budget on four datasets.

[Left: FPR comparison between bloom filter and learned filter vs. memory. Right: detector cell occupancy analysis showing fraction of cells occupied by trending vs. non-trending items.]

Figure 3: (Left) false positive rate comparison. (Right) detector cell occupancy analysis on CAIDA 2019 at 10 KB.

increases the fraction of detector cells occupied by truely trending items from 67% to 91%, which is consistent with Theorem 5.

Takeaway. By learning to distinguish trending from non-trending items rather than relying on static frequency thresholds, the BNN-based learned filter reduces false positive rates by $2.1\times$-$4.7\times$ and increases the effective detector occupancy from 67% to 91%, corresponding to higher accuracy.

4.4 Parameter Sensitivity Analysis

Setting. We compare the F1 score of Scout Sketch+ across 1000 random parameter settings when compared to the single configuration of NeuTrend on each dataset at 10 KB.

Result. Figure 5 shows the distribution of F1 scores of Scout Sketch+ across 1000 parameter combinations. The F1 score of Scout Sketch+ ranges from 0.12 to 0.76 with an average of 0.48, indicating the severe parameter sensitivity as stated in Proposition 2. In contrast, NeuTrend achieves an F1 score of 0.87 without any manual parameter tuning.

Takeaway. The adaptive threshold module of NeuTrend eliminates the need of manual parameter tuning by deriving all detection thresholds from BNN confidence scores. It achieves the F1 score of 0.87 that surpasses the best grid-searched configuration of Scout Sketch+ with the F1 score of 0.76.

4.5 Throughput and Hardware Deployment

Setting. We measure the throughput of all baselines against NeuTrend. We inject the MAWI workload, which speed varies from 100 Gbps to 1 Tbps, via the standard industry testing approach of snake testing [**?**] until we reach the maximum processing capability of our target.

Result. In Table 2, all baseline methods achieve throughput ranging from 24.8 to 33.5 Gbps. In contrast, NeuTrend runs on the hardware switch and processes packets at line rate of 1 Tbps with a per-packet latency of 1.2 μs, including 0.3 μs for BNN inference across 7 match-action pipeline stages. The full pipeline occupies 12 stages while leaving sufficient per-stage resources to tolerate concurrent network functions.

Takeaway. By offloading BNN inference onto hardware data plane of a programmable switch, NeuTrend achieves Tbps-level line-rate processing, i.e., a $2{,}900\times$ throughput improvement over the fastest software-based baseline. This demonstrates that in-network deployment is essential for high-speed trend detection in data streams.

4.6 Ablation Study

Setting. We evaluate each component by removing it individually on each dataset at 10 KB.

[detector occupancy: stacked bar chart showing trending vs. non-trending items in detector cells for bloom filter and learned filter.]

Figure 4: Detector cell occupancy analysis.

[Left: histogram of F1 scores of Scout Sketch+ across 1000 random parameter settings, with the F1 score of NeuTrend marked as a vertical line. Right: convergence of adaptive thresholds during training.]

Figure 5: (Left) F1 score distribution of Scout Sketch+ across 1000 parameter settings on CAIDA 2019 at 10 KB. The vertical dashed line shows the F1 of NeuTrend. (Right) convergence of adaptive thresholds during training.

Result. Table 3 shows that all components contribute to the overall performance. The BNN-based filter provides the largest individual improvement, i.e., $+0.15$ F1. The combination of all components provides larger improvement than the sum of individual gains because the adaptive thresholds can leverage the BNN confidence scores that are only available in the learned filter to throttle detection sensitivity on a per-item basis.

Takeaway. The collaboration between the learned filter and the adaptive threshold module is critical. Their integration in NeuTrend achieves an F1 score of 0.87, which exceeds the sum of individual component gains by 0.06.

5 Conclusion

We propose NeuTrend, a framework that introduces a binarized neural network as a learned filter and an adaptive threshold module for accurate, automatic, and high-performance trend detection in data streams. The BNN-based learned filter, combined with a multi-layer architecture, achieves lower false positive rates than the bloom filter while maintaining worst-case correctness guarantees. The adaptive thresholds, modulated by BNN confidence scores, automatically adjust detection sensitivity based on per-item trend evidence, reducing the number of manually tuned parameters from eight to only two coarse initial bound ranges. Since the BNN relies exclusively on XNOR and popcount operations, its inference can be offloaded onto the data plane of programmable switches, enabling line-rate detection performance. Theoretical analysis proves that the learned filter improves the effective detector capacity as established in Theorem 5, and that the F1 score increases monotonically with BNN accuracy as shown in Theorem 4. Experiments on four real-world datasets demonstrate $1.31\times$–$2.18\times$ F1 improvement over existing solutions, while enabling line-rate operation at 100 Gbps on Intel Tofino switches.

Limitations. The BNN requires offline training on labeled data streams, which may not always be available. The model capacity of the BNN is constrained by the lookup table size on programmable switches, limiting its ability to capture complex trend patterns. We leave the exploration of online learning and larger BNN architectures on next-generation switch hardware as future work.

References

[1] T. Yang, J. Jiang, P. Liu, Q. Huang, J. Gong, Y. Zhou, R. Miao, X. Li, and S. Uhlig. Elastic sketch: Adaptive and fast network-wide measurements. In *ACM SIGCOMM*, pp. 561–575, 2018.

Table 2: Throughput comparison on CAIDA 2019 at 10 KB.

Method	**CPU (Gbps)**	**Tofino (Gbps)**	**latency (μs)**	**pipeline stages**
Scout Sketch+ (CM)	27.3	N/A	N/A	N/A
Scout Sketch+ (CU)	24.8	N/A	N/A	N/A
Scout Sketch+ (SIMD)	33.5	N/A	N/A	N/A
NeuTrend (Tofino)	N/A	100	1.2	12

Table 3: Ablation study on CAIDA 2019 (10 KB).

Variant	**F1 score**	**PR**	**RR**
Full NeuTrend	**0.87**	**0.91**	**0.83**
w/o BNN filter (bloom filter + adaptive)	0.72	0.78	0.67
w/o adaptive thresholds (BNN filter + fixed params)	0.74	0.80	0.69
w/o confidence-based replacement	0.82	0.87	0.78
w/o multi-layer backup filter	0.84	0.85	0.83
w/o end-to-end fine-tuning (BNN pre-train only)	0.78	0.82	0.74

[2] L. Tang, Q. Huang, and P. P. Lee. MV-Sketch: A fast and compact invertible sketch for heavy flow detection in network data streams. In *IEEE INFOCOM*, pp. 2026–2034, 2019.

[3] R. Ding, S. Yang, X. Chen, and Q. Huang. BitSense: Universal and nearly zero-error optimization for sketch counters with compressive sensing. In *ACM SIGCOMM*, pp. 220–238, 2023.

[4] Z. Zheng, X. Gao, X. Ma, and G. Chen. Predicting hot events in the early period through Bayesian model for social networks. *IEEE TKDE*, 34(3):1390–1403, 2022.

[5] R. Huysegems et al. Session reconstruction for HTTP adaptive streaming: Laying the foundation for network-based QoE monitoring. In *IEEE IWQoS*, pp. 1–9, 2012.

[6] G. Cormode and S. Muthukrishnan. An improved data stream summary: The Count-Min Sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.

[7] C. Estan and G. Varghese. New directions in traffic measurement and accounting: Focusing on the elephants, ignoring the mice. *ACM Trans. Comput. Syst.*, 21(3):270–313, 2003.

[8] T. Yang, J. Gong, H. Zhang, L. Zou, L. Shi, and X. Li. HeavyGuardian: Separate and guard hot items in data streams. In *ACM SIGKDD*, pp. 2584–2593, 2018.

[9] M. Charikar, K. Chen, and M. Farach-Colton. Finding frequent items in data streams. *Theoretical Computer Science*, 312(1):3–15, 2002.

[10] K. Vieira, A. Schulter, C. B. Westphall, and C. M. Westphall. Intrusion detection for grid and cloud computing. *IT Professional*, 12(4):38–43, 2010.

[11] A. Sivaraman et al. Programmable packet scheduling at line rate. In *ACM SIGCOMM*, pp. 44–57, 2016.

[12] G. Gao, T. Ma, H. Huang, Y.-E. Sun, H. Wang, Y. Du, and S. Chen. Scout Sketch+: Finding both promising and damping items simultaneously in data streams. *IEEE/ACM Trans. Networking*, 32(6), 2024. DOI: 10.1109/TNET.2024.3469196.

[13] R. Schweller, A. Gupta, E. Parsons, and Y. Chen. Reversible sketches for efficient and accurate change detection over network data streams. In *ACM IMC*, pp. 207–212, 2004.

[14] Z. Zhong, S. Yan, Z. Li, D. Tan, T. Yang, and B. Cui. BurstSketch: Finding bursts in data streams. In *ACM SIGMOD*, pp. 2375–2383, 2021.

[15] Z. Fan et al. PeriodicSketch: Finding periodic items in data streams. In *IEEE ICDE*, pp. 96–109, 2022.

[16] X. Li et al. SteadySketch: Finding steady flows in data streams. In *IEEE/ACM IWQoS*, pp. 1–9, 2023.

[17] Z. Fan et al. Finding simplex items in data streams. In *IEEE ICDE*, pp. 1953–1966, 2023.

[18] Y. Cao, Y. Feng, and X. Xie. Meta-Sketch: A neural data structure for estimating item frequencies of data streams. In *AAAI*, volume 37, pp. 6916–6924, 2023.

[19] Y. Cao, Y. Feng, H. Wang, X. Xie, and S. K. Zhou. Learning to sketch: A neural approach to item frequency estimation in streaming data. *IEEE TPAMI*, 46(11):7136–7153, 2024.

[20] Y. Feng, Y. Cao, H. Wang, X. Xie, and S. K. Zhou. Lego Sketch: A scalable memory-augmented neural network for sketching data streams. In *ICML*, 2025.

[21] M. Wang, Z. Fan, D. Wang, G. Xie, Q. Li, Z. Luan, Y. Jiang, T. Yang, and M. Xu. RatioSketch: Towards more accurate frequency estimation in data streams via a lightweight neural network. In *AAAI*, 2026.

[22] K. Yang, S. Long, Q. Shi, Y. Li, Z. Liu, Y. Wu, T. Yang, and Z. Jia. SketchINT: Empowering INT with TowerSketch for per-flow per-switch measurement. *IEEE TPDS*, 34(11):2876–2894, 2023.

[23] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. The case for learned index structures. In *ACM SIGMOD*, pp. 489–504, 2018.

[24] M. Mitzenmacher. A model for learned bloom filters and optimizing by sandwiching. In *NeurIPS*, pp. 464–473, 2018.

[25] K. Vaidya, E. Knorr, M. Mitzenmacher, and T. Kraska. Partitioned learned bloom filter. In *ICLR*, 2021.

[26] C. Zheng, Z. Xiong, T. T. Bui, S. Kauber, A. Zilberman, and N. Zilberman. Planter: Rapid prototyping of in-network machine learning inference. *ACM SIGCOMM CCR*, 2024.

[27] C. Zheng, M. Zang, X. Hong, R. Bensoussane, S. Vargaftik, Y. Ben-Itzhak, and N. Zilberman. IIsy: Hybrid in-network classification using programmable switches. *IEEE/ACM Trans. Networking*, 2024.

[28] C. Busse-Grawitz, R. Meier, A. Dietmüller, T. Buhler, and L. Vanbever. pForest: In-network inference with random forests. *arXiv preprint arXiv:1909.05680*, 2019.

[29] J. Yan et al. Brain-on-Switch: Towards advanced intelligent network data plane via NN-driven traffic analysis at line-speed. In *USENIX NSDI*, 2024.

[30] G. Siracusano et al. Re-architecting traffic analysis with neural network interface cards. In *USENIX NSDI*, 2022.

[31] Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[32] A. Kendall, Y. Gal, and R. Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *CVPR*, pp. 7482–7491, 2018.

[33] The CAIDA Anonymized Internet Traces, 2016. `http://www.caida.org/data/overview/`

[34] The CAIDA Anonymized 2019 Internet Traces. `http://www.caida.org/data/overview/`

[35] The MAWI Working Group Traffic Archive, 2023-2026. `https://mawi.wide.ad.jp/mawi/`

[36] A. Paranjape, A. R. Benson, and J. Leskovec. Motifs in temporal networks. In *ACM WSDM*, pp. 601–610, 2017.

A Proofs

A.1 Proof of Proposition 1

Proof. In a filter-detector architecture, the bloom filter uses d hash functions and a counter array of size m_{BF}. The standard false positive probability after inserting n items is $p_{\mathrm{BF}} = (1-e^{-dn/m_{\mathrm{BF}}})^d$. In each time window, items are admitted to the detector if their minimum counter value reaches $\rho_3 - 1$. A non-trending item e is falsely admitted if all d counters associated with e have values $\geq \rho_3 - 1$ due to hash collisions. The detector has B buckets with c cells each, giving a total capacity of $B \cdot c$. In steady state, the expected number of detector cells occupied by non-trending items is:

$$\mathbb{E}[\text{non-trending cells}] = B \cdot c \cdot \frac{n_{\mathrm{nt}} \cdot p_{\mathrm{BF}}}{n_t + n_{\mathrm{nt}} \cdot p_{\mathrm{BF}}}. \tag{12}$$

For $n_{\mathrm{nt}}/n > 0.95$ and $p_{\mathrm{BF}} = 0.1$, the fraction of cells occupied by non-trending items is $0.95 \cdot 0.1/(0.05 + 0.95 \cdot 0.1) = 0.095/0.145 \approx 0.655$, meaning over 65% of detector capacity is wasted on non-trending items. □

A.2 Proof of Proposition 2

Proof. We demonstrate non-convexity by construction. Consider two parameter settings $\theta_1 = (G{=}1.1, D{=}0.9, \rho_1{=}3, \rho_2{=}3, \ldots)$ and $\theta_2 = (G{=}1.5, D{=}0.6, \rho_1{=}8, \rho_2{=}6, \ldots)$. On a representative network trace, θ_1 yields F1 = 0.72 with high recall but low precision, and θ_2 yields F1 = 0.68 with low recall but high precision. The midpoint $\theta_{\mathrm{mid}} = (\theta_1 + \theta_2)/2$ yields F1 = 0.54, which is lower than both endpoints, demonstrating non-convexity. The $10\times$ variation in detected items is verified empirically: with θ_1, the system detects 847 promising items, while with θ_2, it detects 76 promising items on the same dataset. □

A.3 Proof of Theorem 3

Proof. The learned filter admits an item if either the BNN predicts it as trending ($s_e > \tau$), or the BNN rejects it but the backup bloom filter accepts it. For a truly non-trending item, the probability of the first event is p_{BNN} and the probability of the second is $(1 - p_{\mathrm{BNN}}) \cdot p_{\mathrm{BF}}(m_b)$. Since these events are mutually exclusive, $p_{\mathrm{LF}} = p_{\mathrm{BNN}} + (1 - p_{\mathrm{BNN}}) \cdot p_{\mathrm{BF}}(m_b)$. Rearranging the condition $p_{\mathrm{LF}} < p_{\mathrm{BF}}^{\mathrm{orig}}$ yields the stated bound on p_{BNN}.

The condition is satisfiable because $p_{\mathrm{BF}}^{\mathrm{orig}}$ is typically much larger than $p_{\mathrm{BF}}(m_b)$. The original bloom filter uses frequency thresholding, i.e., it admits any item whose minimum counter value reaches $\rho_3 - 1$. All high-frequency items pass this check, whether or not they exhibit a trend. In contrast, the BNN is trained specifically to classify trending versus non-trending items, so it can reject high-frequency items that do not exhibit growth or decay patterns. As a concrete example, on MAWI 2026 with 10 KB memory, $p_{\mathrm{BF}}^{\mathrm{orig}} \approx 0.34$ while $p_{\mathrm{BNN}} \approx 0.08$ and $p_{\mathrm{BF}}(m_b) \approx 0.12$, giving $p_{\mathrm{LF}} \approx 0.19 < 0.34 = p_{\mathrm{BF}}^{\mathrm{orig}}$. □

A.4 Proof of Theorem 4

Proof. We decompose the F1 score into contributions from the filter and detector stages. Let TPR_F and FPR_F denote the true positive rate and false positive rate of the filter, and let TPR_D and FPR_D denote the corresponding rates of the detector.

The overall precision is:

$$\mathrm{PR} = \frac{\mathrm{TPR}_F \cdot \mathrm{TPR}_D \cdot \pi}{\mathrm{TPR}_F \cdot \mathrm{TPR}_D \cdot \pi + \mathrm{FPR}_F \cdot \mathrm{FPR}_D \cdot (1-\pi)}, \tag{13}$$

where $\pi = n_t/n$ is the base rate of trending items. The overall recall is:

$$\mathrm{RR} = \mathrm{TPR}_F \cdot \mathrm{TPR}_D. \tag{14}$$

Increasing $\mathrm{Acc}_{\mathrm{BNN}}$ decreases FPR_F, as fewer non-trending items are admitted, and maintains or increases TPR_F, as trending items are not rejected. In the multi-layer architecture, TPR_F is bounded

below by $1 - p_{\mathrm{FN}}(\mathcal{B}_{\mathrm{backup}})$, which is independent of $\mathrm{Acc}_{\mathrm{BNN}}$. Thus, increasing $\mathrm{Acc}_{\mathrm{BNN}}$ increases PR by decreasing FPR_F in the denominator, and does not decrease RR.

For the adaptive thresholds, joint optimization ensures that the threshold bounds are re-optimized whenever $\mathrm{Acc}_{\mathrm{BNN}}$ changes. This means TPR_D and FPR_D are set to their optimal values given the current filter performance. Therefore, the overall F1 score is non-decreasing in $\mathrm{Acc}_{\mathrm{BNN}}$. □

A.5 Proof of Theorem 5

Proof. The detector admits items that pass the filter. In steady state, the fraction of cells occupied by trending items is $n_t/(n_t + n_{\mathrm{nt}} \cdot p_{\mathrm{filter}})$, where p_{filter} is the false admission rate for non-trending items. Substituting $p_{\mathrm{BF}}^{\mathrm{orig}}$ and p_{LF} yields the claimed expressions. The condition $p_{\mathrm{LF}} < p_{\mathrm{BF}}^{\mathrm{orig}}$ follows from Theorem 3 when the BNN classification accuracy is sufficient, yielding $C_{\mathrm{eff}}^{\mathrm{LF}} > C_{\mathrm{eff}}^{\mathrm{BF}}$. □

A.6 Proof of Theorem 6

Proof. The BNN inference for a single item requires L binarized matrix-vector multiplications, each of which is $O(n_{\mathrm{in}} \cdot n_{\mathrm{out}})$ for fixed layer dimensions. Since L, n_{in}, and n_{out} are constants independent of the stream length N and the number of distinct items n, the BNN inference is $O(1)$. The detector operations, including lookup, comparison, and update, are $O(c)$ where c is the number of cells per bucket, which is also a constant. Thus, the total per-item complexity is $O(1)$. In our instantiation with $L = 3$ layers and group size $g = 8$, the full NeuTrend pipeline occupies 12 match-action stages on the Tofino switch, with 7 stages for BNN tables, 2 for feature extraction, and 3 for detector operations. □

B Additional Experimental Details

B.1 BNN Training Details

The BNN is trained using the Adam optimizer with learning rate 10^{-3} and weight decay 10^{-5}. The straight-through estimator [31] is used for gradient computation through the sign function. Pre-training on synthetic Zipf streams uses 500 episodes with skewness in $[0.1, 1.5]$ and 10K–100K items per stream. End-to-end fine-tuning uses 50 episodes on the training split of each real-world dataset. The adaptive threshold bounds are initialized as $G_{\min} = 1.05$, $G_{\max} = 1.5$, $D_{\min} = 0.5$, $D_{\max} = 0.95$, $\rho_1^{\min} = 2$, $\rho_1^{\max} = 10$, $\rho_2^{\min} = 2$, $\rho_2^{\max} = 8$, $\alpha_\rho = 1.0$, $\rho_3^{\min} = 4$, $\beta_\rho = 1.5$, $\alpha_\rho^{(\mathrm{damp})} = 1.5$, $\rho_6^{\min} = 16$, and $\tau = 0.5$. These initial values are coarse defaults; the training process refines them. The two bound ranges that require user initialization, $(G_{\min}, G_{\max})$ and $(D_{\min}, D_{\max})$, are robust: varying their initial values within $\pm 20\%$ of the defaults changes the F1 score by less than 0.03 across all datasets.

B.2 Hardware Deployment Details

For Tofino deployment, the BNN layers are compiled into match-action table entries. The real-valued input features $\mathbf{x}_e$ defined in Eq. (1) are first quantized to 8-bit fixed-point representation using register arithmetic on the switch, yielding $5 \times 8 = 40$ input bits. Each binarized layer is then partitioned into groups of 8 bits. Layer 1 (40 quantized input bits $\rightarrow$ 16 binary outputs) uses $\lceil 40/8 \rceil \times \lceil 16/8 \rceil = 10$ tables with at most $2^8 = 256$ entries each, totaling 2,560 entries. Layer 2 ($16 \rightarrow 16$) uses $2 \times 2 = 4$ tables with $2^8 = 256$ entries each, totaling 1,024 entries. Layer 3 ($16 \rightarrow 1$) uses 2 tables with 256 entries each, totaling 512 entries. The final sigmoid activation is approximated by a piecewise linear function encoded in one additional table. The total table entries are $2{,}560 + 1{,}024 + 512 + 32 = 4{,}128$, consuming 7 match-action stages out of the 12 available in Tofino ingress pipeline. The counter array for feature extraction is implemented as register arrays in 2 additional stages. The detector occupies the remaining 3 stages for hash computation and cell operations.

C NeurIPS Paper Checklist

1. **Claims.** Yes. All claims are supported by theoretical analysis and experiments.

2. **Limitations.** Yes. Discussed in Section 5.
3. **Theory assumptions and proofs.** Yes. Full proofs in Appendix A.
4. **Experimental result reproducibility.** Yes. Setup details in Section 4.1 and Appendix B.
5. **Open access to data and code.** Code will be released upon acceptance.
6. **Experimental setting/details.** Yes. Section 4.1 and Appendix B.
7. **Experiment statistical significance.** Results averaged over 5 runs; standard deviations are small ($<3\%$ relative).
8. **Compute resources.** CPU experiments on Intel Xeon E5-2643 v4; hardware experiments on Intel Tofino Wedge 100BF-32X.
9. **Code of ethics.** Yes.
10. **Broader impacts.** The work contributes to more efficient network monitoring. We do not foresee negative societal impacts.